SMART FARMING

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

Y.UMESH REDDY | AP23110011667

M.MAHESH CHOWDARY | AP23110011665

G.SRIRAM | AP23110011663

M.KALYAN | AP22110011633

Under the Guidance of

Ms.V.Veda Sri

Lecturer, Department of CSE

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

[APRIL, 2025]

Certificate

Date: 25-04-2025

This is to certify that the work present in this Project entitled "**SMART FARMING**" has been carried out by **[Our Team]** under my/our supervision. The work is genuine, original, and suitable for submission to the SRM University – AP for the award of Bachelor of Technology/Master of Technology in **School of Engineering and Sciences**.

Supervisor

Kavitha Rani

Designation,

Affiliation.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, **KAVITHA RANI**, Department of Computer Science & Engineering, SRM University, Andhra pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work.

M.MAHESH CHOWDARY-AP23110011665

Y.UMESH REDDY-AP23110011667

G.SRIRAM - AP23110011663

M.KALYAN -AP22110011633

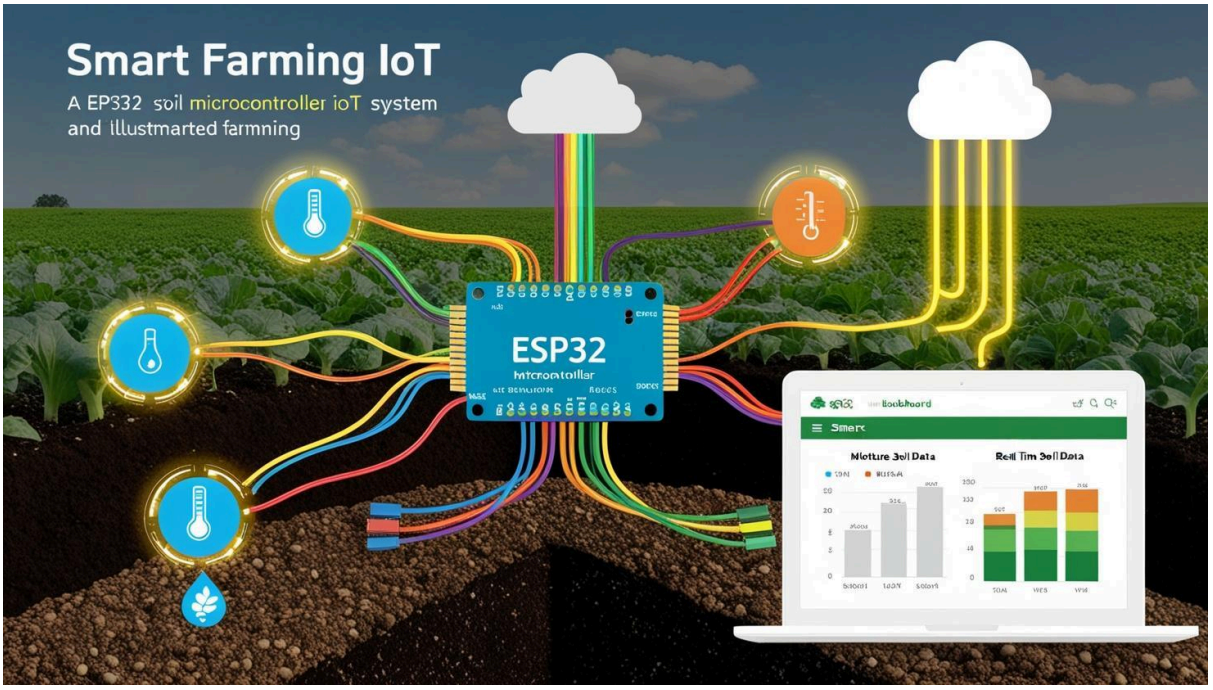
Table of Contents

Abstract

Smart farming is an innovative approach to modern agriculture that integrates advanced technologies like the Internet of Things (IoT) to improve the efficiency, productivity, and sustainability of farming practices. This project focuses on developing an IoT-based soil monitoring system designed to assist farmers in making informed decisions based on real-time environmental data. The primary problem addressed is the lack of timely and accurate information on soil conditions, which often leads to inefficient irrigation and reduced crop yields.

The system employs an ESP32 microcontroller connected to various sensors to measure soil moisture, temperature, and pH levels. These sensors continuously collect data, which is then transmitted to a cloud-based platform for monitoring and analysis. A web dashboard displays the data in real-time and alerts the user when critical thresholds are crossed. Additionally, the system can trigger automatic watering based on soil moisture levels, reducing water wastage and manual labor.

The key findings demonstrate that the system effectively monitors soil conditions, enabling timely interventions and improving water usage. In conclusion, this smart farming solution has the potential to support more sustainable agricultural practices and can be expanded further for broader applications, such as precision farming and crop health monitoring.



1. Introduction

Project Background:

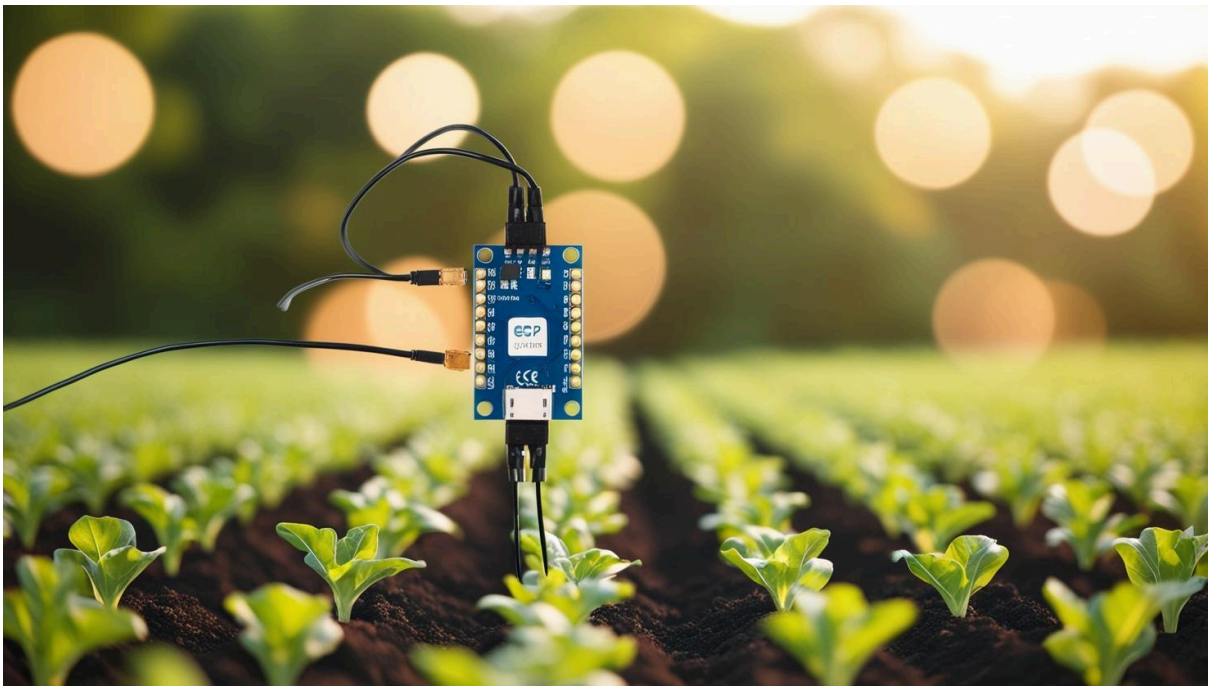
Agriculture has always been the backbone of human civilization, providing food, raw materials, and employment for billions worldwide. However, traditional farming methods are increasingly challenged by climate change, water scarcity, soil degradation, labor shortages, and the growing demand for food due to rapid population growth. To address these issues, the agricultural sector is undergoing a significant transformation—enter **Smart Farming**.

By adopting smart farming solutions, farmers can make data-driven decisions, reduce resource wastage, optimize yields, and minimize environmental impact. This digital revolution is not just modernizing agriculture; it is redefining the future of food production, ensuring food security while promoting sustainable farming practices for generations to come.

1.1. Significance and context:

The significance of smart farming lies in its potential to revolutionize agriculture by addressing the key challenges faced by modern-day farmers. As global populations continue to rise, the demand for food is expected to increase by 60% by 2050, placing enormous pressure on existing agricultural systems. At the same time, climate change, land degradation, and water scarcity threaten the reliability and sustainability of traditional farming practices.

In this context, smart farming emerges as a critical solution. By leveraging data and automation, it enables more accurate and efficient farming operations. Sensors can monitor soil moisture levels, drones can survey crop health, and AI can predict pest outbreaks or suggest the best planting times. These innovations reduce resource usage, lower production costs, and increase crop yields, helping to ensure food security while conserving the environment.



1.1.1. Scope and Purpose:

The purpose of this project is to explore the concept, applications, and benefits of smart farming technologies in modern agriculture. It aims to analyze how the integration of digital tools and intelligent systems can enhance productivity, reduce environmental impact, and improve decision-making processes in farming practices.

This project specifically focuses on the following key areas:

- **Technological Components of Smart Farming**: Examining the role of IoT devices, sensors, drones, data analytics, AI, and robotics in agricultural operations.

- **Benefits and Impact**: Evaluating how smart farming improves efficiency, optimizes resource usage, supports sustainability, and contributes to food security.

- **Challenges and Limitations**: Identifying barriers such as high initial costs, technological complexity, limited connectivity in rural areas, and the need for

technical training.

- **Real-World Applications and Case Studies**: Reviewing successful implementations of smart farming in various regions and agricultural contexts.

- **Future Prospects**: Exploring emerging trends and the future potential of smart farming in addressing global agricultural demands.

The **scope** of this project is limited to crop farming and resource management (such as soil, water, and climate monitoring) within the context of smart agriculture. While livestock farming and aquaculture also benefit from smart technologies, they are not the primary focus of this report.

This project is intended for an audience interested in agricultural innovation, including students, researchers, policy makers, and agribusiness stakeholders, with the goal of raising awareness about the transformative power of smart farming and encouraging its wider adoption.

2. Methodology

Approach:

The project follows a hands-on, application-based approach aimed at solving a real-world agricultural problem—overwatering or underwatering crops due to a lack of accurate soil condition data. The main goals were to monitor soil parameters, automate irrigation, and visualize data for easy decision-making.

Methods:

1. **Soil Monitoring**: Sensors were placed in the soil to measure key parameters such as moisture, pH, and temperature.

2. **Data Collection**: Sensor readings were collected at regular intervals and transmitted to a central system.

3. **Data Storage**: All sensor data was stored in a database for real-time access and historical analysis.

4. **Dashboard Display**: A web dashboard was developed to visualize real-time soil conditions, providing users with intuitive insights.

5. **Automation**: The system automatically triggered a watering system whenever the soil moisture dropped below a predefined threshold.

Tools and Technologies:

ESP32: This is a small and powerful microcontroller used to collect data from soil sensors. It also connects to Wi-Fi, sends data to the cloud, and controls the watering system.

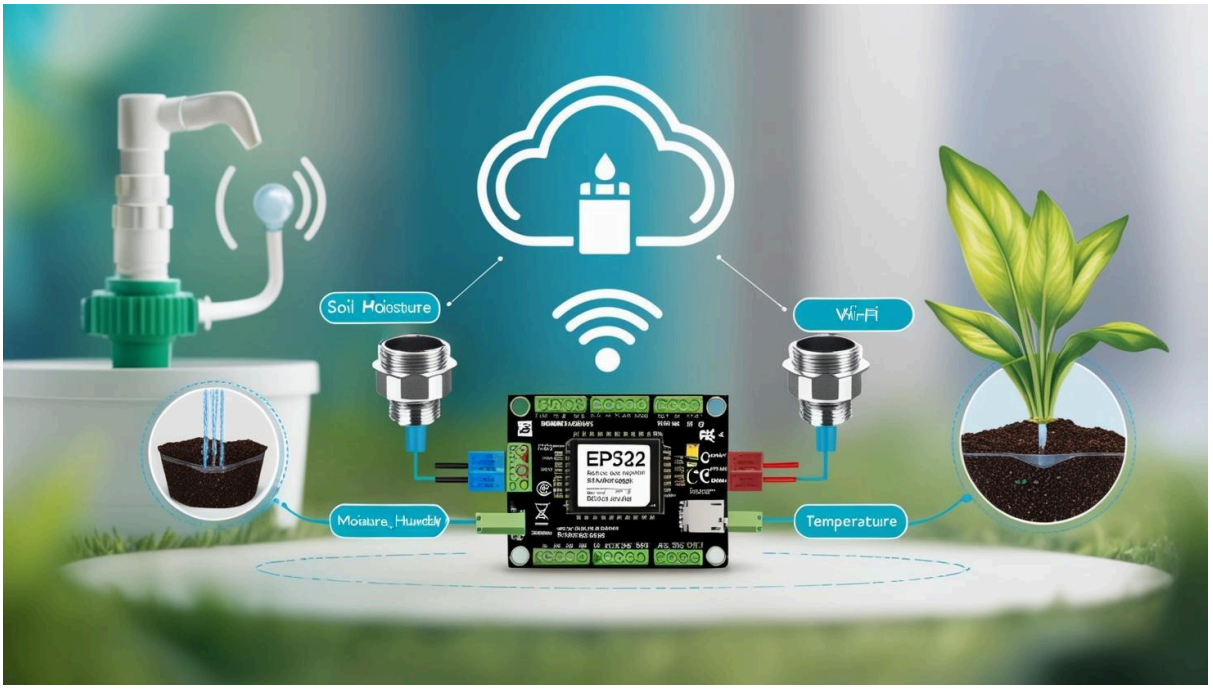
Soil Sensors:

- **Moisture Sensor** – Checks how wet or dry the soil is.
- **Humidity Sensor** – Monitors the air humidity around the plants.
- **Temperature Sensor** – Tells the temperature of the soil.

Wi-Fi: The ESP32 uses Wi-Fi to send the sensor data to the cloud.

Cloud Platform: The cloud is used to store the sensor data and show it on a web dashboard. This helps users see the data from anywhere.

Automatic Watering System: When the moisture in the soil is too low, the ESP32 automatically turns on the water system to keep the soil healthy.



2.1 Data Collection, Experimental Setup, Software, and Algorithms:

Data Collection:

In this project, data is collected using sensors that measure key soil parameters:

- **Soil Moisture Sensor**: Measures the water content in the soil to determine if irrigation is needed.
- **Soil Temperature Sensor**: Measures the temperature of the soil, which affects plant growth and can help optimize irrigation schedules.
- **Humidity Sensor**: Monitors the air humidity around the plants, which can affect soil moisture evaporation rates and plant health.

The **ESP32 microcontroller** collects the data from these sensors at regular intervals. It then sends this data to the cloud, where it is stored for future analysis. This data is also displayed on a web dashboard for real-time monitoring.

Experimental Setup:

- **ESP32** is connected to the sensors that measure soil moisture, pH, and temperature.

- The ESP32 is also connected to a **Wi-Fi network** to transmit the collected data to a cloud platform.

- The system includes an **automated irrigation system** that is triggered by the ESP32 when the soil moisture falls below a specific threshold. This ensures that crops receive the correct amount of water without wasting resources.

Software Used:

- The project uses **cloud platforms** (such as Firebase or a similar service) for storing sensor data and displaying it on a **web-based dashboard**. The dashboard allows users to view real-time data about soil conditions from anywhere.

- The code running on the **ESP32**. This controls the sensors, processes the data, and triggers the irrigation system when necessary.

Algorithms

- The core algorithm is based on a **threshold-based system**. It compares the soil moisture reading to a predefined threshold:

    - If moisture is below the set level, the irrigation system is triggered.

    - If moisture is above the threshold, the system does nothing and waits for the next reading.

2.1.1 Justification for Chosen Methods:

ESP32 Microcontroller: Chosen for its built-in Wi-Fi capabilities and ability to handle multiple sensors. It's inexpensive, energy-efficient, and supports cloud communication for real-time data transfer.

Cloud Storage: Using the cloud allows for easy access to data from anywhere, makes it scalable, and provides a reliable way to store large amounts of sensor data over time.

Automated Irrigation: Ensures water is only used when necessary, reducing water waste and promoting sustainability, which is critical in agriculture.

Threshold Algorithm: Simple and effective for the task at hand. It minimizes complexity while ensuring the system reacts quickly to changes in soil conditions.

Table:

Smart Farming Dashboard

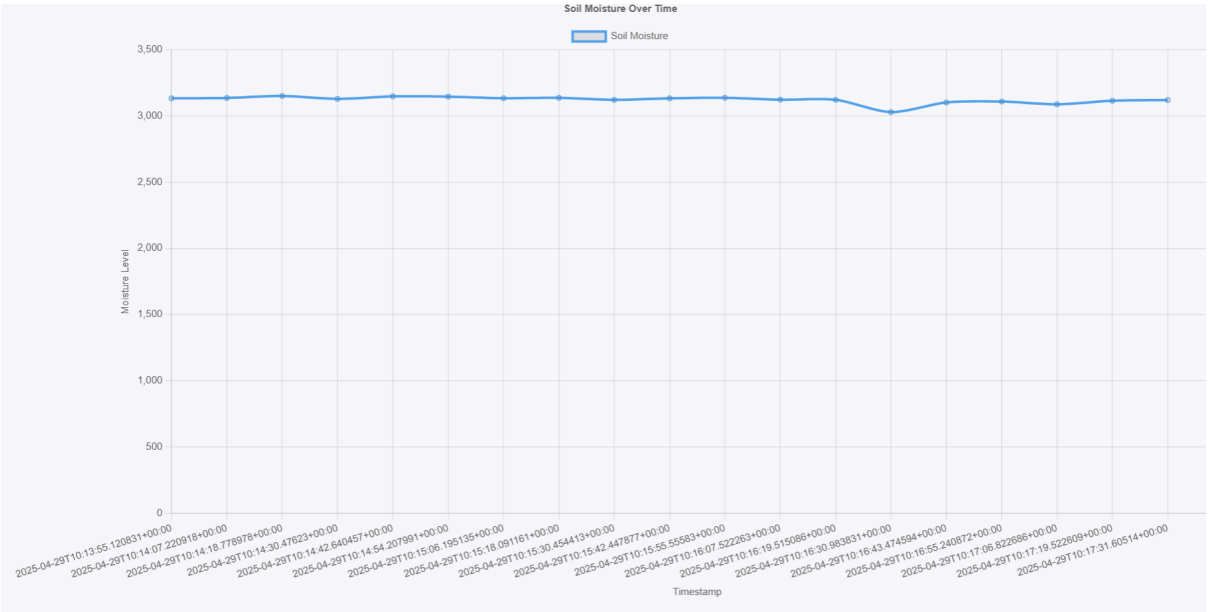
Timestamp	Soil Moisture	Temperature (°C)	Pump Status
2025-04-29T10:13:55.120831+00:00	3133	29.4375	False
2025-04-29T10:14:07.220918+00:00	3136	29.375	False
2025-04-29T10:14:18.778978+00:00	3151	29.375	False
2025-04-29T10:14:30.47623+00:00	3129	29.3125	False
2025-04-29T10:14:42.640457+00:00	3148	29.3125	False
2025-04-29T10:14:54.207991+00:00	3146	29.25	False
2025-04-29T10:15:06.195135+00:00	3134	29.25	False
2025-04-29T10:15:18.091161+00:00	3137	29.1875	False
2025-04-29T10:15:30.454413+00:00	3122	29.1875	False
2025-04-29T10:15:42.447877+00:00	3133	29.125	False
2025-04-29T10:15:55.55583+00:00	3137	29.0625	False
2025-04-29T10:16:07.522263+00:00	3123	29.0625	False
2025-04-29T10:16:19.515086+00:00	3122	29.0625	False
2025-04-29T10:16:30.983831+00:00	3029	29	False
2025-04-29T10:16:43.474594+00:00	3102	28.9375	False

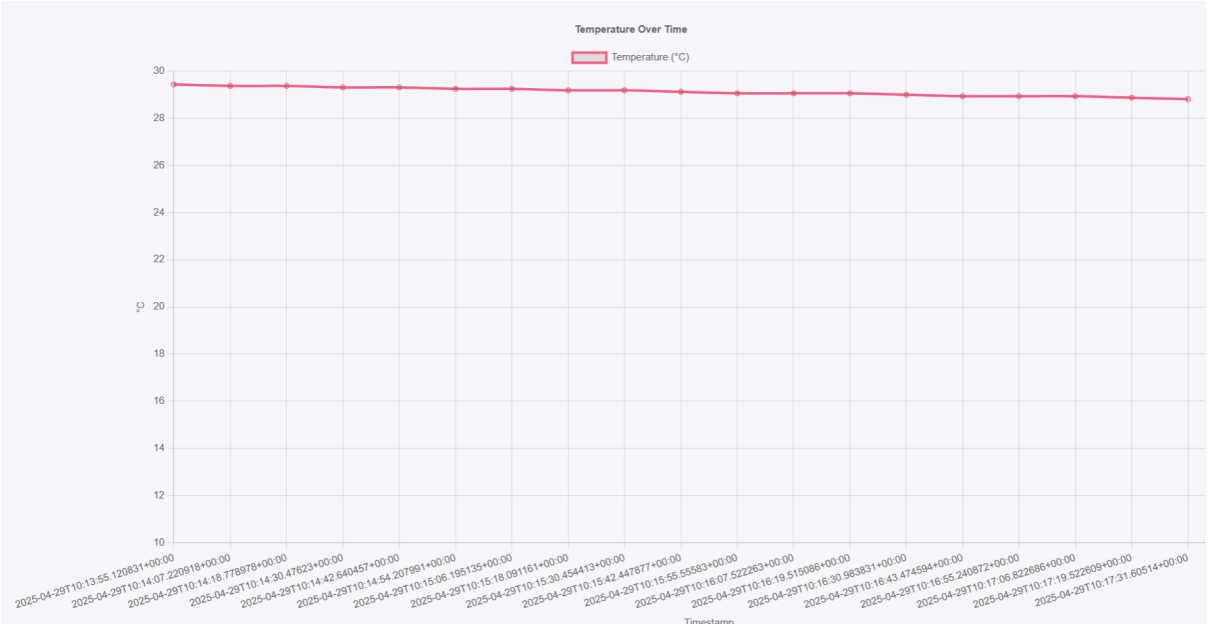
Time represents different times or periods at which the sensors take measurements.

Temperature Sensor measures the soil or air temperature at each time point.

Moisture Sensor measures the soil moisture level.

Humidity Sensor measures the air humidity.





The current environmental conditions appear **suitable for healthy plant growth**, but **irrigation may be needed soon** if the soil moisture continues to drop. Regular monitoring ensures optimal crop yield and prevents stress due to dryness or excess water.

3. Implementation

Steps to Implement the Smart Farming:

1. Problem Identification:

Agriculture often faces inefficiencies in water usage and manual monitoring.

To solve this, we aimed to develop a smart, automated system for soil condition monitoring and irrigation using IoT.

2. Planning and Component Selection:

Selected ESP32 for its Wi-Fi capabilities and GPIO support.

Chose sensors:

DHT11/DHT22 for temperature and humidity.

Soil Moisture Sensor for detecting dryness levels.

Added a Relay Module to control a Water Pump.

Decided on MicroPython with Thonny IDE for programming due to ease of use and compatibility with ESP32.

3. Hardware Setup Using Breadboard:

All components were connected on a breadboard to create a temporary and test-friendly circuit.

The DHT sensor was connected to a digital GPIO (e.g., GPIO 4) via the breadboard.

Soil Moisture Sensor was connected to an analog GPIO (e.g., GPIO 34).

The Relay Module was connected to a digital GPIO (e.g., GPIO 12), allowing control of the water pump.

Proper power and ground connections were established using the breadboard rails.

ESP32 was powered through USB from a laptop or power bank.

4. MicroPython Firmware Installation:

Downloaded and flashed MicroPython firmware to the ESP32 using esptool.py:

esptool.py --port COMx erase_flash

esptool.py --port COMx --baud 460800 write_flash -z 0x1000 esp32-xxxx.bin

Verified successful installation via Thonny's serial monitor.

5. Programming with Thonny:

Opened Thonny IDE, selected interpreter as MicroPython (ESP32).

Wrote code to:

Read data from the DHT and moisture sensors.

Compare moisture value with a defined threshold.

Activate relay (and thus the pump) if moisture is too low.

Uploaded the script to ESP32, saving it as main.py for automatic execution on boot.

6. Testing and Calibration:

Observed real-time sensor readings through Thonny's shell.

Calibrated the moisture threshold by testing dry vs. wet soil.

Verified the relay module successfully switched the pump based on conditions.

Ensured all connections were stable on the breadboard.

7. Deployment and Observation:

The setup was tested in real conditions using potted plants or garden soil.

System operated continuously, watering plants only when needed.

The breadboard setup helped in quick troubleshooting and modifications.

9. Conclusion and Future Work:

The prototype successfully demonstrated real-time soil monitoring and automated irrigation.

Future enhancements could include:

Replacing breadboard with PCB for permanent use.

Adding more sensors for multiple zones.

Building a custom mobile app for control and alerts.

Code snippets:

Micropython:

```
import network
import requests
import time
from machine import ADC, Pin
import dht
# Wi-Fi credentials
SSID = "Net Ho Gaya?"
PASSWORD = "Mahi@665"
# Replace with your PC's IP and port
FLASK_SERVER = "http://192.168.188.21:5000/api/submit"
# Sensor setup
soil = ADC(Pin(34))  # Soil moisture sensor
soil.atten(ADC.ATTN_11DB)
dht_sensor = dht.DHT11(Pin(4))  # DHT11 temperature sensor
relay = Pin(2, Pin.OUT)  # Relay for water motor
# Thresholds
MOISTURE_THRESHOLD = 2000
TEMP_THRESHOLD = 30
def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(SSID, PASSWORD)
    while not wlan.isconnected():
        print("Connecting to Wi-Fi...")
```

```
        time.sleep(1)
    print("Connected to Wi-Fi:", wlan.ifconfig())


def send_data(moisture, temp, pump_status):
    try:
        data = {
            "moisture": moisture,
            "temperature": temp,
            "pump_status": pump_status
        }
        print("Sending:", data)
        res = urequests.post(FLASK_SERVER, json=data)
        print("Response:", res.text)
        res.close()
    except Exception as e:
        print("Failed to send:", e)
# Start Wi-Fi connection
connect_wifi()
# Main loop
while True:
    try:
        dht_sensor.measure()
        temp = dht_sensor.temperature()
    except Exception as e:
        print("DHT error:", e)
        temp = -1
```

```
    moisture = soil.read()
    print("Moisture:", moisture, "Temp:", temp)

    if moisture > MOISTURE_THRESHOLD and temp >
TEMP_THRESHOLD:
        relay.value(1)
        pump_status = "ON"
    else:
        relay.value(0)
        pump_status = "OFF"

    send_data(moisture, temp, pump_status)
    time.sleep(10)
print(wlan.ifconfig())
```

Flask:

```
from flask import Flask, request, render_template, jsonify
import requests
import datetime

app = Flask(__name__)

# Supabase Configurations
SUPABASE_URL = "https://stvavzacncijxinqbzhv.supabase.co"
SUPABASE_API_KEY =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSI
sInJlZiI6InN0dmF2emFjbmNpanhpbnFiemh2Iiwicm9sZSI6ImFub24iLCJp
YXQiOjE3NDI0Njg2NTMsImV4cCI6MjA1ODA0NDY1M30.RIqxh5HHfIbeXe4sZ
LTOc6AfPHq_6EPTGnpMzNXxUdQ"
SUPABASE_TABLE = "data"
```

```
HEADERS = {
    "apikey": SUPABASE_API_KEY,
    "Authorization": f"Bearer {SUPABASE_API_KEY}",
    "Content-Type": "application/json"
}

@app.route('/')
def dashboard():
    try:
        res = requests.get(

f"{SUPABASE_URL}/rest/v1/{SUPABASE_TABLE}?order=time_stamp.de
sc&limit=20",
            headers=HEADERS
        )
        if res.status_code == 200:
            data = res.json()
            if isinstance(data, list):
                data.reverse()
            else:
                print("Unexpected data format from Supabase")
                data = []
            return render_template("dashboard.html",
records=data)
        else:
            return jsonify({"error": "Failed to fetch data
from Supabase"}), 500
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route('/api/submit', methods=["POST"])
def submit():
    try:
        incoming = request.json
        if "temperature" not in incoming or "moisture" not in
incoming or "pump_status" not in incoming:
            return jsonify({"error": "Missing required
data"}), 400

        # Add timestamp
        incoming["time_stamp"] =
datetime.datetime.utcnow().isoformat()

        # Post the data to Supabase
        res = requests.post(
            f"{SUPABASE_URL}/rest/v1/{SUPABASE_TABLE}",
            headers=HEADERS,
            json=[incoming]
        )
```

```
        if res.status_code in [200, 201]:
            return jsonify({"status": "ok"})
        else:
            return jsonify({"error": "Failed to insert
data"}), 500
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    app.run(host="0.0.0.0", port=5000, debug=True)
```

4. Result and Analysis

Overview:

The Smart Farming System integrates IoT sensors, data analytics, and automated systems to monitor and manage agricultural operations. This project focused on real-time monitoring of soil moisture, temperature, humidity, and crop health to improve efficiency and productivity in farming.

Data collection:

Sensor Type	Parameter	Unit	Sampling Interval
DHT11	Temperature	°C	Every 1 day
DHT11	Humidity	%	Every 1 day
Soil Moisture	Moisture	%	Every 1 day

Smart farming sensors enable real-time data collection on soil, crops, and environmental conditions.

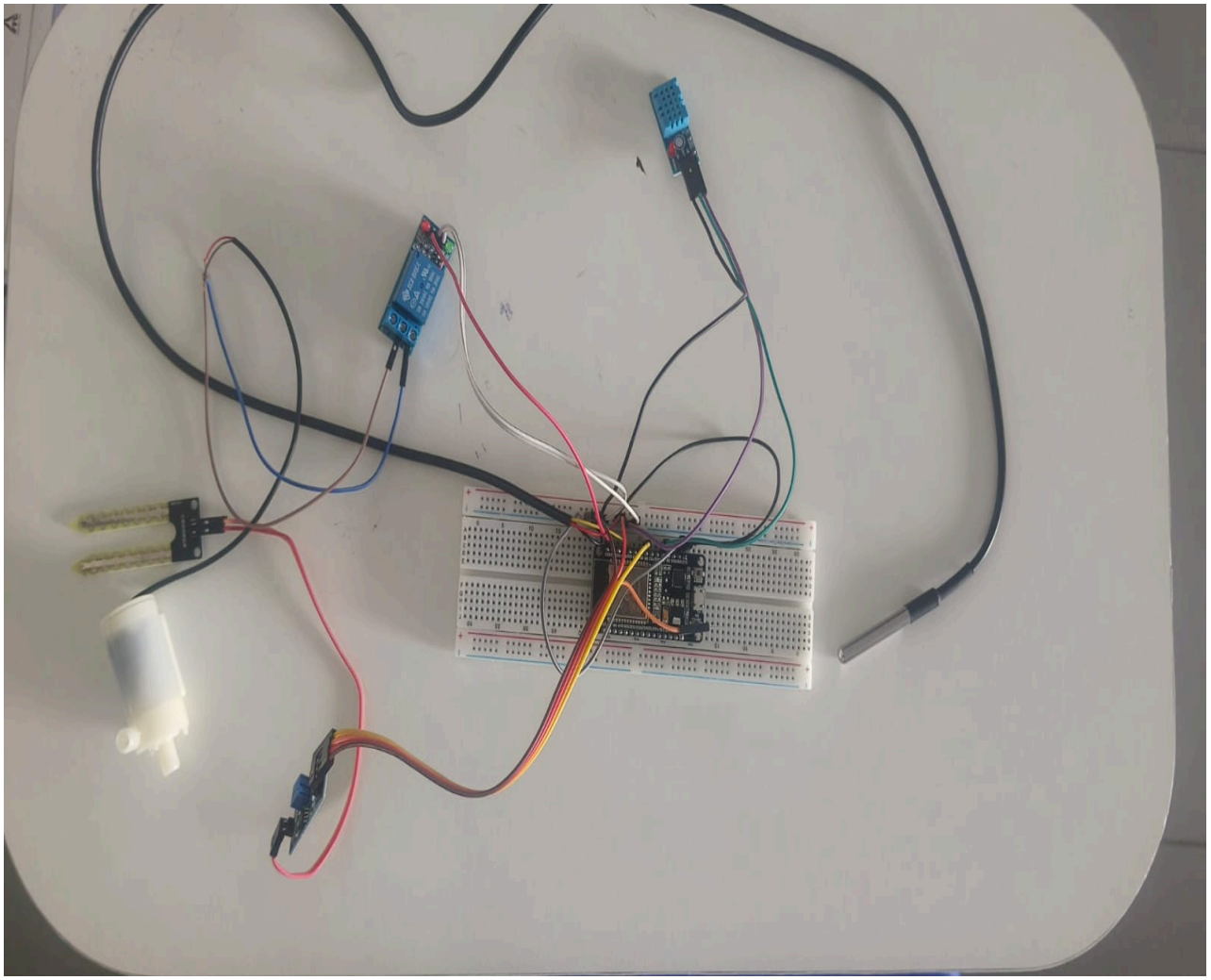
This data helps farmers make informed, timely decisions to boost productivity and sustainability.

Sensors improve resource efficiency by optimizing water, fertilizer, and pesticide usage.

They also support remote monitoring and automation, reducing labor and increasing accuracy.

Despite their benefits, challenges like connectivity and cost must be managed for broader use.

Overall result:



5. Discussion and Conclusion

Main Points:

This project looked at how smart farming sensors collect data about soil, crops, and weather. It showed that using sensors helps farmers make better decisions and work more efficiently.

Why It Matters:

The results prove that sensors can help save water, reduce waste, and grow healthier crops, making farming more sustainable.

Project Contribution:

This project helps improve modern farming by showing how technology can make farming smarter and more effective.

Overall conclusion:

Smart farming uses modern technology like sensors to improve how farming is done.

This project showed how sensors can collect real-time data on soil, weather, and crops.

The data helps farmers make better decisions and take quick action when needed.

It leads to better crop health, higher yields, and less use of water and chemicals.

Smart farming also saves time and reduces the need for manual labor.

Remote monitoring and automation make it easier to manage large farms.

The project proved that smart farming is more efficient and eco-friendly.

It also pointed out some challenges like high costs and internet access in rural areas.

Still, the benefits of using technology in farming are clear and promising.

This project adds to the growing field of precision agriculture and smart farm solutions.

6. Future Scope

1. How You Can Improve the Current System

- **Use Better Sensors**
  Use strong and waterproof sensors that give more accurate results and last longer.

- **Save More Power**
  Use solar panels or batteries to run the system without needing constant electricity.

- **Improve Internet Connection**
  If Wi-Fi is weak on the farm, use SIM cards or long-range communication to send data.

2. Add More Features in the Future

- **Store Data for Longer**
  Save all temperature, humidity, and moisture data in a database so you can see changes over days or months.

- **Use Weather Forecasts**
  Add weather data to your system so it can decide not to water the plants if it's going to rain.

- **Get Alerts on Your Phone**
  Create a mobile app that shows the data and sends you alerts (like "Soil is dry" or "Pump turned on").

3. Smarter Automation with AI

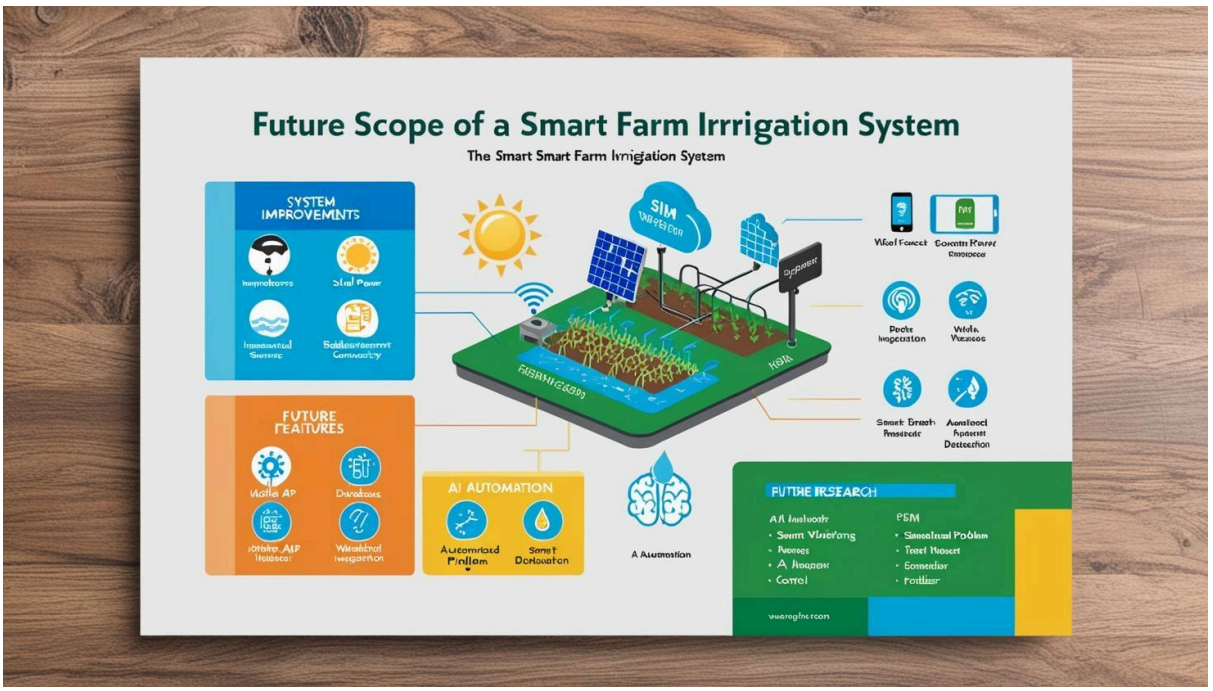
- **Smart Decisions with AI**
  Train the system to learn when and how much to water plants based on past data and weather.

- **Detect Problems Automatically**
  The system can find if a sensor is giving wrong data or if a plant is not doing well.

- **Smart Nutrient Control**
  Add fertilizer delivery to the system so it can give nutrients when the plant needs it.

4. Ideas for Future Research

- Study how soil nutrients (pH, salt) affect plant growth.
- Use drones or cameras to monitor plant health.
- Expand the system for large farms or different types of crops.
- Try to make a full automatic farming system (irrigation, lighting, nutrients).



References

1. Rani, S., Ahmed, S. H., & Malrey, L. (2020). *Smart farming: An approach for precision agriculture using IoT and cloud computing*. In Smart Sensor Networks,Springer.

2. **MicroPython. (n.d.). Installing MicroPython on the ESP32.** Retrieved from https://docs.micropython.org/en/latest/esp32/quickref.html